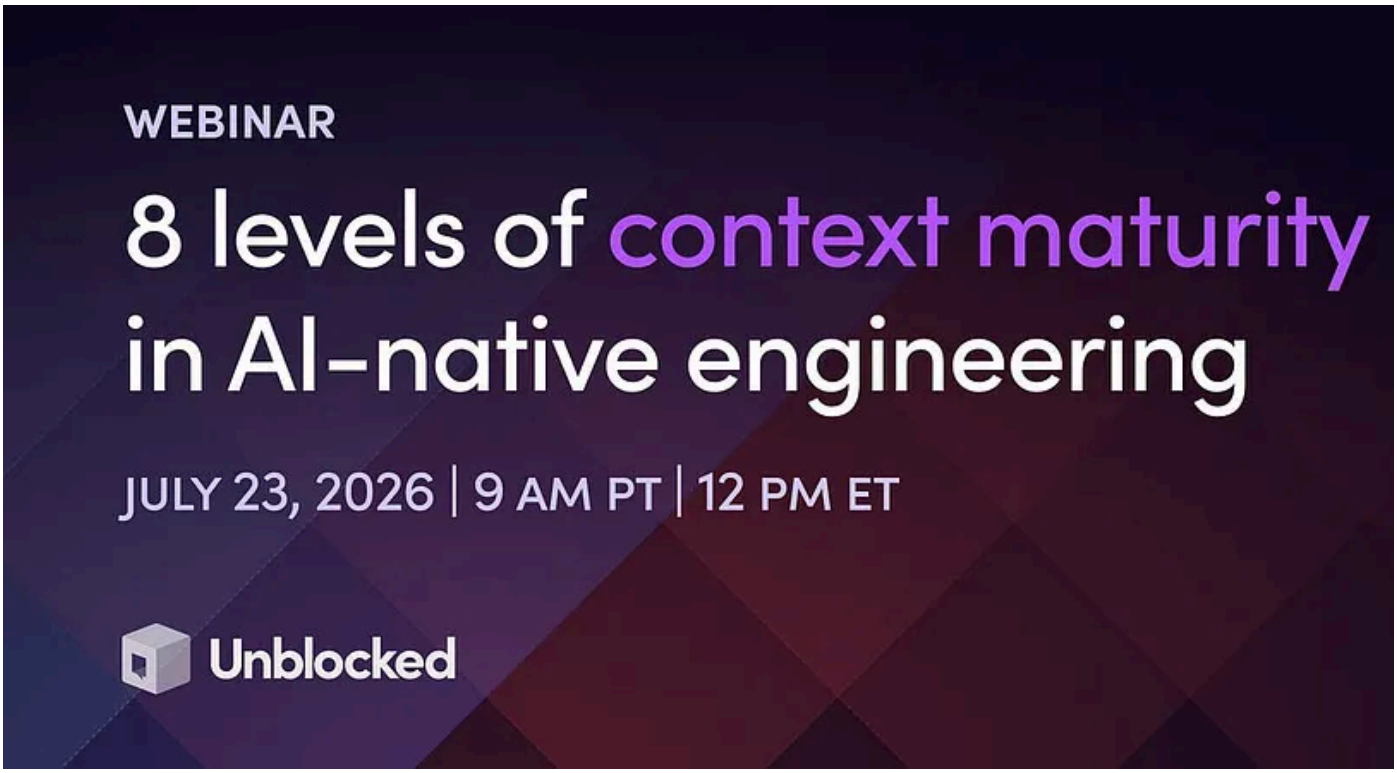
Best Practices for Building AI Agents That Work in Production

BYTEBYTEGO
JUL 22, 2026

[Webinar] 8 levels of context maturity i AI-native engineering (Sponsored)



AI is in your engineering workflows. While the token spend shows it, the throughp doesn't. The human is very much still in the loop, and that's a context problem.

Join live July 23 (FREE) to learn:

- The 8 levels of context maturity: where most teams are stuck and what the ceili
  looks like at each stage
- Why more MCPs, rules, and skills provide agents access but not understanding
- How leading en he most of
  their agents

There is a persistent monstrati
and the ones that ho

This is because the s pend on th
language model far l ehavior ru
through conventiona all numbe
of specific decision nd a fragil
one often comes dov model is
given control and wh

But how do you buil any best
practices?

In 2011, Heroku's co-founder published the Twelve-Factor App, a set of principles f
building web services that deploy and scale reliably, and that document influenced
how a generation of engineers thought about software.

A parallel effort later appeared for AI in the form of a widely shared set of principle
called the Twelve-Factor Agents. These principles were aimed at making language-
model applications robust enough for production. The specifics have kept evolving
since, and teams at companies like Anthropic, Cognition, and Intercom have added
lessons of their own.

In this article, we try to explore the collective thinking into a smaller set of practice and explain the reasoning behind each one, rather than asking anyone to memorize numbered list. The practices fall into four areas, with a final section on the tradeoff Here's what we will cover:

- How does context control what the model sees on every call?
- How does the control flow keep the loop and its stopping conditions in deterministic code?
- How does the state hold memory in software while the model stays stateless?
- How does the scope keep each agent narrow and supervised?

By the end, the article will have built a working definition of a production agent an clear view of which decisions carry the most weight.

Disclaimer: This post is based on publicly shared details from various sources. References a the end. Please comment if you notice any inaccuracies.

The Loop

Let us start with the simplest agent anyone can build, because it reveals the structu
that everything else refines.

At its core, an agent is a loop. The model receives some context, which is the runni
record of what has happened so far, and it returns a single decision about the next s
to take. That decision arrives as structured output. It is machine-readable data, suc
JSON rather than free-flowing prose, so that ordinary code can read it and act on it
The surrounding code executes the requested step, appends the result to the contex
and returns everything to the model for the next decision. The loop continues until
model signals that the work is finished.

A useful way to picture the model's role is as a function that starts fresh on every ca

On each call, the model sees only the context provided to it, makes one decision, an
discards everything the moment it responds. The context window, which is the bun
of text the model reads on a given call, serves as the entire memory of the system.

The appeal of this design is clear. A developer describes a goal and supplies a set of tools, and the model works out the order of operations on its own. The design requi less branching logic, since the model handles the decisions. This is the version of a agent that can perform quite well in a demo setup. However, the trouble begins wh real traffic arrives.

Failure Modes

The design we looked at in the previous section fails under production conditions, a the failures follow some predictable patterns:

- Compounding error multiplies across chained steps.
- Confidently wrong output reaches a user.
- A loop runs longer than intended.
- State disappears when the plan lives only inside the model.

Let's understand them in a little more detail.

The first pattern is compounding error. Every model call carries some chance of a wrong decision, and a freewheeling agent chains many calls together. Suppose each step is correct 95 percent of the time, which sounds reliable. Run twenty such steps sequence, and the odds that all of them succeed fall to roughly one in three. The ma is multiplicative, so reliability that looks fine in isolation degrades quickly across a long chain. This fact explains why production teams add so many guardrails.

The second pattern is confidently wrong output reaching a user. In 2025, the suppo agent for the coding tool Cursor told customers about a one-device-per-subscriptio rule that the company later confirmed it had fabricated, and a wave of cancellations followed. A year earlier, a tribunal held Air Canada liable after its chatbot invented bereavement-fare policy, and ordered the airline to honor it. Studies place hallucination rates, meaning the rate at which a model produces plausible but false statements, somewhere between 3 and 27 percent, even in controlled settings. An agent that speaks directly to customers turns that rate into a big business risk.

The final two patterns are harder to notice, but just as costly.

A loop with weak stopping conditions can run far longer than intended, consuming tokens and money with each pass. And lastly, when the plan lives only inside the model's context, a crash or a long-running task can lose the entire thread of work.

Each failure points to a specific practice, grouped into the four areas below, beginn with the inputs.

Context

The context window is everything the model knows on a given call, so controlling i contents is the first and largest lever on reliability. Three habits do most of the wor here.

The first is to own your prompts. A prompt is the instruction text sent to the model and it deserves the same treatment as any other source code, which means version control, review, and testing. Many agent frameworks generate prompts automatical and keep them out of sight, which is convenient until behavior changes and the cau is hard to locate. Keeping prompts in the application's own codebase keeps their effects reproducible.

The second habit is to own the context window itself. The contents of each call are deliberate choice, and that choice carries more weight than it first appears. A mode given a focused, relevant context performs better than the same model handed a lar pile of loosely related history. Quality degrades as the window fills with marginal material. Therefore, deliberate pruning by actively removing content beyond what t current step requires preserves the model's accuracy. In other words, relevance beat volume on almost every call.

The third habit concerns tools. A tool is simply a function the model can request, described with a clear name and a schema for its inputs. When those descriptions a precise, the model selects the right tool and fills its arguments correctly. When they are vague, the model is more likely to choose the wrong one. Treating tool definitio as a careful piece of interface design pays off immediately.

A discipline called context engineering has grown up around these habits, and it's a big topic in itself. However, the core idea stays simple: decide what the model sees, purpose, every time.

Control Flow

Owning the inputs matters only if the surrounding code decides when the model ru and when the loop ends. In a dependable agent, the control flow belongs to the

deterministic code around the model, and the model is consulted at a few chosen moments.

Picture the overall flow as ordinary code, a sequence of steps, conditionals, and call external systems. At two or three points in that flow, where a decision genuinely ca for judgment, the system invokes the model. Everywhere else, plain deterministic lo does the work because it is cheaper to run, predictable in its output, and straightforward to test.

Two practical rules make it work:

- First, every loop needs an escape hatch, which is a hard limit that forces it to st A cap on the number of iterations, a timeout, and explicit completion conditio together guarantee the agent halts even when the model would continue indefinitely.
- Second, invoke the model on purpose. The model belongs where the problem requires open-ended reasoning, and ordinary code handles everything that can specified in advance.

Intercom's customer service product shows the pattern at scale.

Its newer Procedures let the agent reason in natural language while the surroundin system supplies deterministic controls, including conditional steps for decision poi small code snippets that guarantee the same input always yields the same output, a checkpoints where the agent pauses for human approval before a sensitive action.

To summarize, it is beneficial to start with the simplest version that solves the problem, and add model-driven autonomy only where simpler logic falls short.

State

Once the surrounding code owns the loop, a natural question follows. Where does t agent's memory live?

The answer is to keep the model stateless and to hold the state in software that the application controls.

Recall that the model starts each call fresh and reads only the context provided to it That property, when treated as a feature, becomes powerful. The application stores real state of the work, the conversation so far, the plan, the progress, and reconstru the context from it on every call. Since the state lives in serializable storage (a form system can save and reload), an agent can pause partway through a task and resume later, or recover cleanly after a crash, by loading the saved state and continuing.

This habit also keeps two kinds of states aligned. There is the model's view of what happening, and there is the application's record of ground truth, such as the actual contents of a database. Keeping these two in agreement prevents the model from acting on a stale or inaccurate view of the current state.

There is a scaling benefit as well.

When the state lives entirely outside the model, any running instance of the service can pick up any request, because everything needed is available with the context. T allows many copies to run behind a load balancer, handling far more traffic.

A clean way to think about the model here is as a function that takes the current st and a new event and returns the next state along with an action to perform. The sar inputs produce the same result, which makes the whole system easier to test and to trust.

Scope

Owning the inputs, the loop, and the state still leaves one decision open. How mucl should a single agent try to accomplish? The good answer favors small, focused age

kept under supervision over one broad agent asked to do everything.

A narrow agent with a single, well-defined job has a smaller surface for failure. It is easier to test, easier to reason about, and easier to fix when it misbehaves. When a t spans several distinct jobs, several narrow agents compose under deterministic orchestration. The surrounding code decides which agent runs and when, rather th assigning one general agent a long list of responsibilities.

Klarna's customer service assistant illustrates the upside. It handled roughly 2.3 million conversations in its first month, and most of those followed predictable pat such as checking a refund status or tracking an order, which structured logic handl well, with the model reserved for the cases that truly need it.

Supervision is the other half of the scope.

A human handoff deserves to be designed as a first-class step. It is reached on purp when an action is sensitive, when confidence is low, or when a customer asks for a person. Intercom builds this in directly, handing a conversation to a human automatically whenever continuing would be the riskier choice. Treating human involvement as a planned path, complete with the state needed to brief the person, turns it into a strength of the system rather than a sign that something went wrong

Tradeoffs

Everything so far describes practices that work today. But there are also disagreeme
and a few open questions.

The main debate concerns single versus multiple agents.

In June 2025, the team behind the coding agent Devin argued against multi-agent designs because parallel sub-agents make independent decisions that then conflict with one another. One day later, Anthropic described its own multi-agent research system, which used roughly fifteen times the tokens of a single-agent approach and scored about 90 percent higher on certain research tasks.

The apparent contradiction resolved over the following months into a shared patte
A single orchestrator owns the full context and spawns isolated, short-lived sub-
agents, each of which completes one task and returns a summary. The lesson to
understand is that small, focused agents work best under firm orchestration, while
sub-agents that communicate directly with one another tend to produce conflicting
results.

The deeper question comes from what Rich Sutton called the Bitter Lesson. It is th
observation that general methods that rely on more computation tend to overtake
handcrafted techniques over time. Applied here, it warns that some of the scaffoldi
built to compensate for today's models will become redundant as the models impro
That is partly true, and it is the strongest argument against over-engineering.

The counterpoint is that several of these problems persist regardless of model
strength. A finite context window, the need to keep a figure on page five of a
document consistent with the same figure on page eight hundred, the requirement
pause and resume safely, all of these survive better models.

Cost becomes another consideration for every decision. Autonomy and multi-agent designs spend more tokens, sometimes far more, so they earn their place only wher the value of the task justifies the bill.

Conclusion

A production agent, reduced to its essentials, is mostly deterministic software that calls a language model at a few deliberate points. The design decisions lie in choosi those points and limiting how much the model decides on its own.

The path to that definition ran through the simplest possible agent and its predicta failures, with each failure answered by one practice:

- Control what the model sees.
- Own the loop and give it a hard stop.
- Hold the state in software while the model stays stateless.
- Keep each agent narrow and supervised.

Around these sit real tradeoffs, including the open argument over multiple agents a the steady improvement in models that will retire some of this work over time. A sound approach starts with the simplest design that solves the problem, measures where it falls short, and grants the model more autonomy only where that autonom provides clear value.

References:

- The Twelve-Factor App
- 12-Factor Agents — HumanLayer
- Anthropic — Building Effective Agents
- Anthropic — How We Built Our Multi-Agent Research System

- Cognition — Don’t Build Multi-Agents
- Intercom — What’s New with Fin 3
- Rich Sutton — The Bitter Lesson
- Cursor support-bot incident — eWeek
- Air Canada chatbot ruling — AI Business

288 Likes • 11 Restacks

Discussion about this post

Comments Restacks

Write a comment...

Oli 1h

built a few agents for my digital products business and the loop pattern here is spot
the ones that survived production were the ones where I kept the model's decision s

as small as possible. every time I let the model decide "what to do next" instead of j "which of these 3 things to do" it broke within a week.

LIKE REPLY